

PROJECT DOCUMENTATION

TrafficVision AI:

Smart Traffic Prediction & Congestion Management System

PROJECT GUIDE

~ Ms.Shravya U

TEAM MEMBERS

- 1.Ch.Ushas shri
- 2.AmitHalder
- 3.Asvitha J

1. Abstract

Urban traffic congestion is a growing challenge for cities, leading to increased travel time, fuel wastage, and reduced quality of life. TrafficVision AI addresses this problem by providing an AI-powered traffic prediction and congestion management platform that helps city authorities monitor traffic conditions, forecast congestion, and optimize traffic flow using real-time and historical traffic data. The system integrates live traffic monitoring, AI/ML-based congestion and peak-hour forecasting, smart route analysis, an alert and notification system, and analytics dashboards, all accessible through a centralized web/mobile application. The backend is built using Python (FastAPI/Flask) with a React.js/Next.js frontend, PostgreSQL. The outcome is a fully functional, end-to-end smart traffic management system suitable for smart cities, traffic control departments, and transportation agencies.

Urban traffic congestion is a growing challenge for modern cities, leading to increased travel time, fuel wastage, air pollution, and reduced quality of life. TrafficVision AI addresses this problem by providing an AI-powered traffic prediction and congestion management platform that helps city authorities monitor traffic conditions, forecast congestion, and optimize traffic flow using real-time and historical traffic data. The system integrates real-time traffic monitoring, AI/ML-based congestion prediction, peak-hour forecasting, smart route analysis, alerts and notifications, and analytics dashboards through a centralized web/mobile application.

The system analyzes traffic data to identify congestion-prone areas, recognize traffic patterns, and predict future traffic conditions. It can also suggest alternative routes based on current traffic conditions, helping reduce vehicle concentration on congested roads. The alert and notification module provides timely information about traffic congestion, road incidents, and predicted peak-hour conditions, while the analytics dashboard presents traffic statistics and trends to support faster, more informed decisions by traffic authorities.

2. Introduction

2.1 Background

As cities grow, road networks face increasing vehicle density, resulting in frequent congestion, longer commute times, and higher accident risks. Traditional traffic management relies heavily on manual monitoring and static signal timings, which cannot adapt to real-time conditions. Smart city initiatives increasingly rely on AI and IoT-driven systems to monitor, predict, and manage traffic more efficiently, improving urban mobility and transportation planning.

2.2 Problem Statement

City authorities and commuters lack a centralized, intelligent system that combines real-time traffic monitoring with predictive analytics. This makes it difficult to anticipate congestion, plan alternate routes in advance, and respond quickly to accidents or road closures, resulting in inefficient traffic flow and avoidable delays.

2.3 Objectives

- Build an AI-powered platform to monitor traffic conditions in real time.
- Predict congestion levels and peak-hour traffic using historical and live data.
- Provide smart route analysis and alternate route recommendations.
- Deliver traffic analytics and reporting through a centralized dashboard.
- Implement an alert and notification system for congestion, accidents, and road closures.
- Support smart city traffic management, transportation agencies, and urban mobility initiatives.

3. Scope

3.1 In Scope

- User management with admin authentication, traffic operator login, and role-based access control.
- Real-time traffic monitoring: vehicle density tracking, congestion monitoring, road utilization analysis.
- AI-based traffic prediction: congestion prediction, peak-hour forecasting, delay estimation, trend analysis.
- Route analysis: alternate route suggestions, route optimization, travel time estimation, road condition monitoring.
- Alert & notification system: congestion, accident, road closure, and emergency traffic alerts.
- Analytics dashboard: traffic reports, congestion heatmaps, road performance tracking, historical insights.
- Deployment using Docker and cloud platforms (AWS/Azure).

3.2 Out of Scope

- Direct control or automation of physical traffic signals/hardware.
- Integration with law enforcement or automated ticketing systems.
- Autonomous vehicle communication (V2X) systems.

4. Existing System & Proposed Solution

4.1 Existing System

Most existing traffic management approaches rely on manual monitoring via CCTV, static/fixed-timing traffic signals, and periodic historical reporting. These systems are largely reactive rather than predictive, cannot process real-time data at scale, and offer limited route guidance or automated alerting during congestion or incidents.

4.2 Proposed Solution

TrafficVision AI is a centralized, AI-powered platform that ingests real-time and historical traffic data (traffic sensors/CCTV feeds, traffic APIs, weather data) and applies machine learning models to predict congestion and peak-hour traffic. It provides traffic authorities and operators with live dashboards, automated alerts, and smart route recommendations, enabling proactive rather than reactive traffic management.

4.3 Key Features

- Authentication and role-based access control.
- Live traffic monitoring and congestion tracking workflows.
- AI-based traffic forecasting and congestion prediction models.
- Smart route analysis and alternate route recommendation.
- Alert and notification workflows for traffic warnings and emergencies.
- Analytics dashboards for traffic trends and congestion analysis.
- Containerized deployment on cloud platforms.

5. Technology Stack

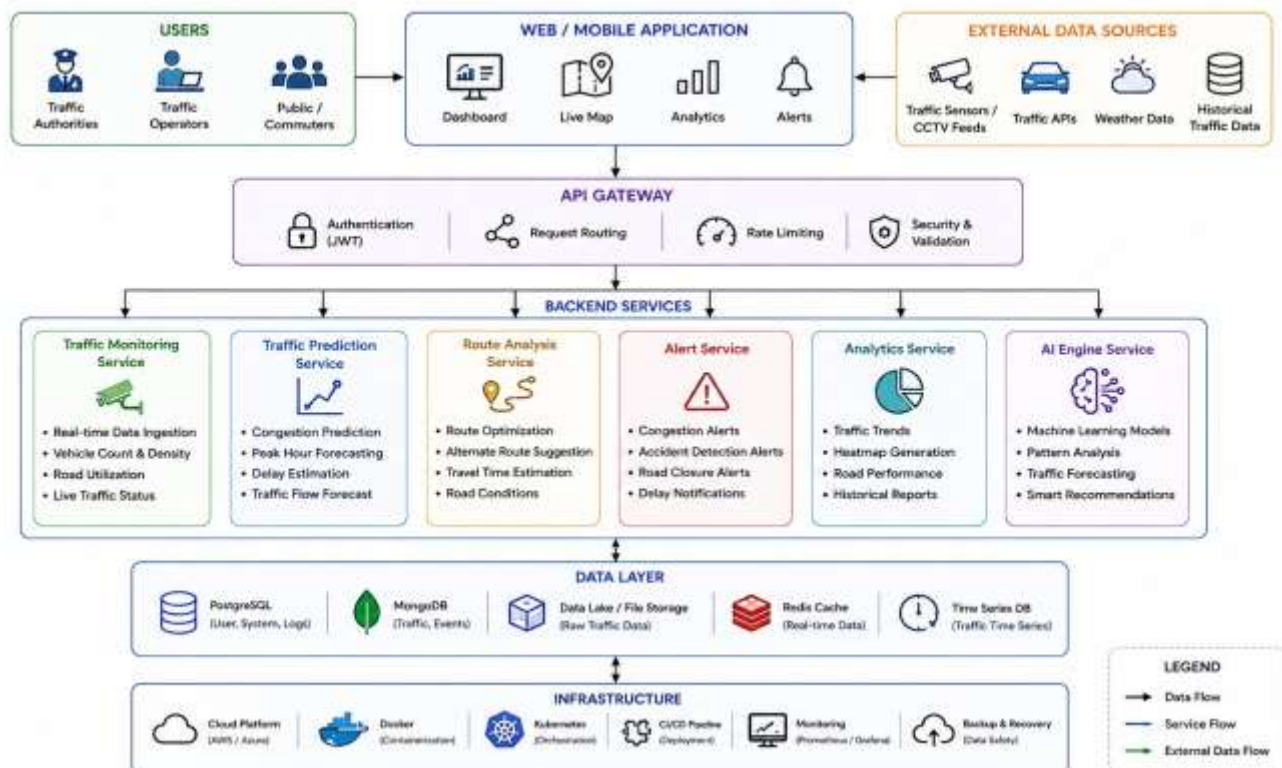
Languages	Python, JavaScript
Frontend	React.js, Next.js, Vite
Backend	Node.js, Express.js, Python — FastAPI
Database	PostgreSQL
AI/ML	Python, Scikit-learn, Pandas, NumPy, Random Forest
Libraries / APIs	Google Maps API, Postman, Newman
Authentication & Security	JWT Authentication, bcrypt password hashing
Frontend Testing	Vitest
Development Tools	Visual Studio Code
Version Control	Git and GitHub
API Communication	REST APIs
Database Management	PgAdmin

6. System Architecture

High-level data flow: **Users (Traffic Authorities / Operators / Public)**, **Web/Mobile Application**, **API Gateway**, **Backend Services**, **AI Engine**, **Data Layer**, **Infrastructure**. External data sources (traffic sensors/CCTV feeds, traffic APIs, weather data, historical traffic data) feed into the backend services, which process and store data for the AI models to generate predictions and recommendations surfaced back through the dashboard.

-The architecture consists of five layers: (1) the **Web/Mobile Application** providing dashboard, live map, analytics, and alerts views; (2) the **API Gateway** handling authentication (JWT), request routing, rate limiting, and security validation; (3) **Backend Services** — Traffic Monitoring, Traffic Prediction, Route Analysis, Alert, Analytics, and AI Engine services; (4) the **Data Layer** using PostgreSQL, MongoDB, a data lake/file storage, Redis cache, and a time-series database; and (5) **Infrastructure** comprising the cloud platform, Docker, Kubernetes, CI/CD pipeline, monitoring, and backup & recovery.

TrafficVision AI – Traffic Prediction System Architecture



6.1 System Modules

User Management Module: Admin authentication, traffic operator login, role-based access control, profile management.

Traffic Monitoring Module: Vehicle density tracking, congestion monitoring, road utilization analysis, live traffic dashboard.

Traffic Prediction Module: Congestion prediction, peak-hour forecasting, delay estimation, traffic trend analysis.

Route Analysis Module: Alternate route suggestions, route optimization, travel time estimation, road condition monitoring.

Alert & Notification Module: Congestion alerts, accident notifications, route delay warnings, emergency traffic alerts.

Analytics Dashboard Module: Traffic analytics reports, congestion heatmaps, road performance tracking, historical traffic insights.

AI Prediction Module: Traffic forecasting models, vehicle flow prediction, smart traffic recommendations, pattern analysis.

7.AI/ML Implementation

7.1 Dataset

The TrafficVision AI system uses real-world traffic data for congestion prediction and analysis. The project team used real Bangalore traffic data to train and evaluate the baseline congestion prediction model. The dataset contains traffic-related information used to identify different levels of congestion and support traffic monitoring and prediction.

The dataset was used as the basis for developing **the Random Forest congestion classification model**. The target variable represents different congestion levels, including **Low, Moderate, High, and Severe conditions**.

The use of real traffic data helps the system model realistic traffic conditions and supports the development of an AI-based approach for congestion detection and prediction.

7.2 Data Processing

The traffic dataset was processed before being used for machine learning. The preprocessing workflow prepares the traffic records and relevant features for congestion classification. The data is organized according to traffic observations and the target congestion category.

The processed data is used to train the Random Forest model for classifying traffic conditions into **Low, Moderate, High, and Severe** congestion levels. The preprocessing and data-splitting workflow also maintains the chronological order of traffic observations to support a reliable train-test evaluation.

7.3 Model / Algorithm

A **Random Forest Classifier** was selected as the baseline machine learning model for traffic congestion prediction. The model is trained using the processed Bangalore traffic dataset and classifies traffic conditions into Low, Moderate, High, and Severe congestion levels.

The Random Forest approach was selected because it can handle multiple traffic-related features and provides a suitable classification approach for identifying different congestion conditions. The model was further regularized during development to reduce overfitting and improve its ability to generalize to unseen traffic data.

The trained model is integrated into the traffic prediction workflow and is used to support congestion analysis and traffic-management decision

7.4 Training & Evaluation

The processed Bangalore traffic dataset was divided into training and testing datasets using **a chronological 80:20 train-test split**. The data was arranged according to the sequence of traffic observations before splitting, helping to maintain the temporal order of the data and reduce the possibility of temporal data leakage during evaluation.

A Random Forest Classifier was trained using the processed traffic data to classify congestion conditions into Low, Moderate, High, and Severe categories. During model development, regularization techniques were applied to reduce overfitting and improve the model's ability to generalize to previously unseen traffic observations.

After training, the model was evaluated using the separate testing dataset. The evaluation focused on classification performance using accuracy and macro F1-score. The final model achieved a test accuracy of 89.3% and a macro F1-score of 0.79. The accuracy indicates that the model correctly classified a high proportion of the test observations, while the macro F1-score provides an overall measure of performance across the different congestion classes.

The evaluation results indicate that the trained Random Forest model provides a strong baseline for traffic congestion classification. The model can support the TrafficVision AI prediction workflow by identifying congestion conditions and providing information that can be used for traffic monitoring, alerts, analytics, and traffic-management decisions.

The trained model is retained for use within the project's traffic prediction workflow, allowing the system to apply the learned classification patterns to traffic data during prediction and analysis.

7.5 AI/ML Workflow

Input - Processing - Model - Output: Raw traffic/sensor data , cleaning & feature extraction, prediction model (congestion/peak-hour forecasting), congestion level, forecast, and route recommendations surfaced on the dashboard.

The resulting congestion predictions support the wider TrafficVision AI workflow by providing information for traffic monitoring, congestion analysis, alerts, analytics, and traffic-management decisions. This workflow enables the system to transform traffic data into useful AI-based congestion insights.

8 Database & API

8.1 Database Schema and ER Diagram

9.1.1	PostgreSQL	User accounts, system data, and logs.
9.1.2	MongoDB	Traffic data and event records.
9.1.3	Data Lake / File Storage	Raw traffic data
9.1.4	Redis Cache	Real-time data caching.
9.1.5	Time Series DB	Traffic time-series data for forecasting.

The TrafficVision AI system uses PostgreSQL as its primary database. The schema contains seven main tables: users, locations, traffic_data, predictions, routes, route_locations, and alerts. These tables are connected through primary-key and foreign-key relationships to maintain data consistency.

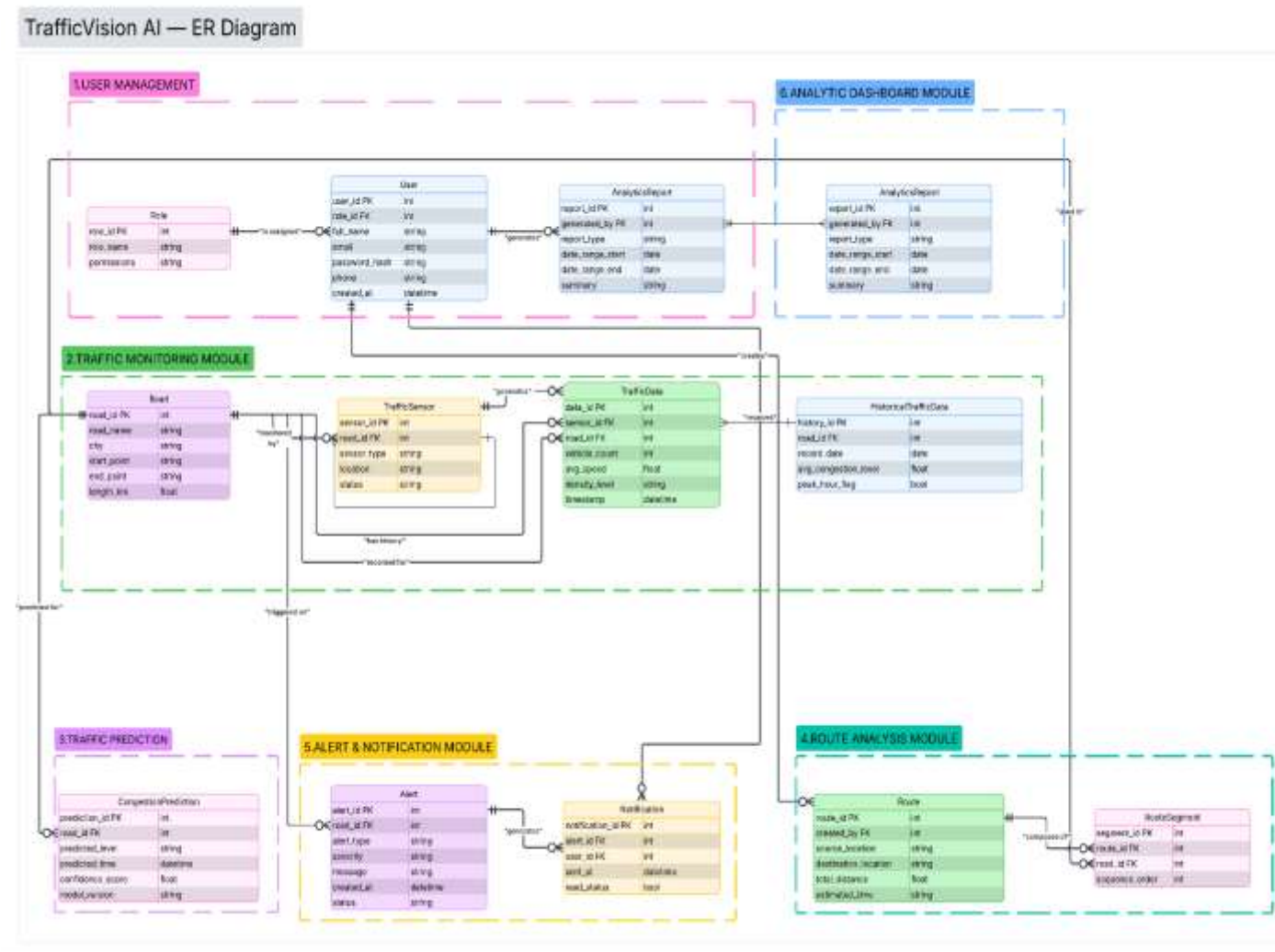
8.2 DATA PROCESSING

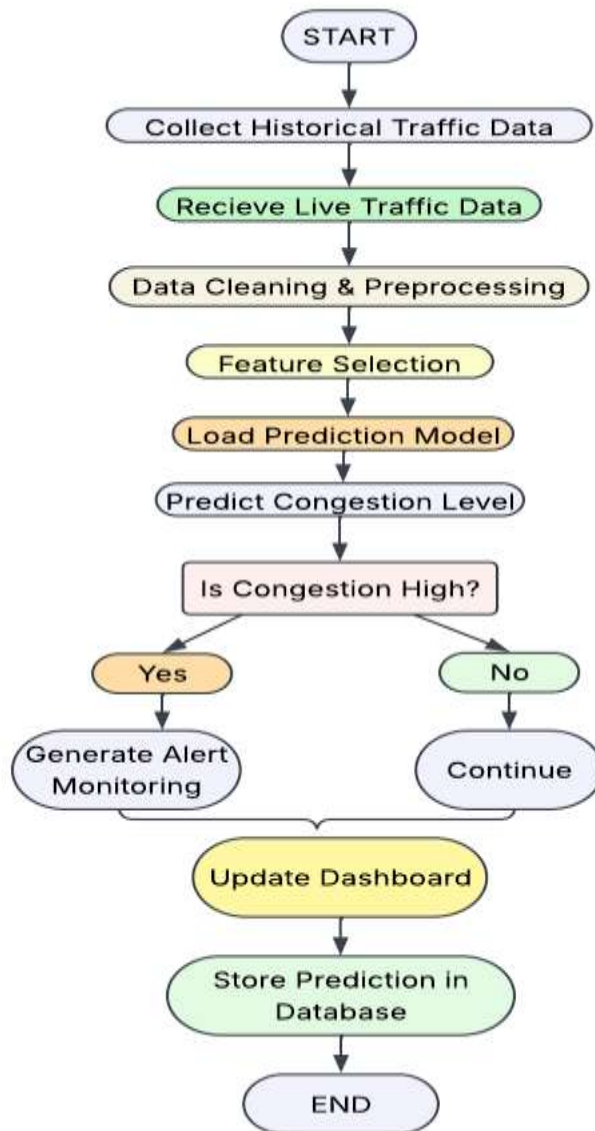
The traffic dataset was cleaned and prepared before model training. The preprocessing stage involved organizing the traffic records, handling the required data fields, and preparing the relevant features for analysis. The data was then arranged chronologically to maintain the correct sequence of traffic observations.

After preprocessing, the prepared data was divided into training and testing sets using an 80:20 chronological split. This processed dataset was then provided as input to the machine learning training workflow.

8.3 ER Diagram:

The ER diagram represents these entities and their relationships, providing the database structure required for traffic monitoring, congestion predictions, route analysis, and alert management.

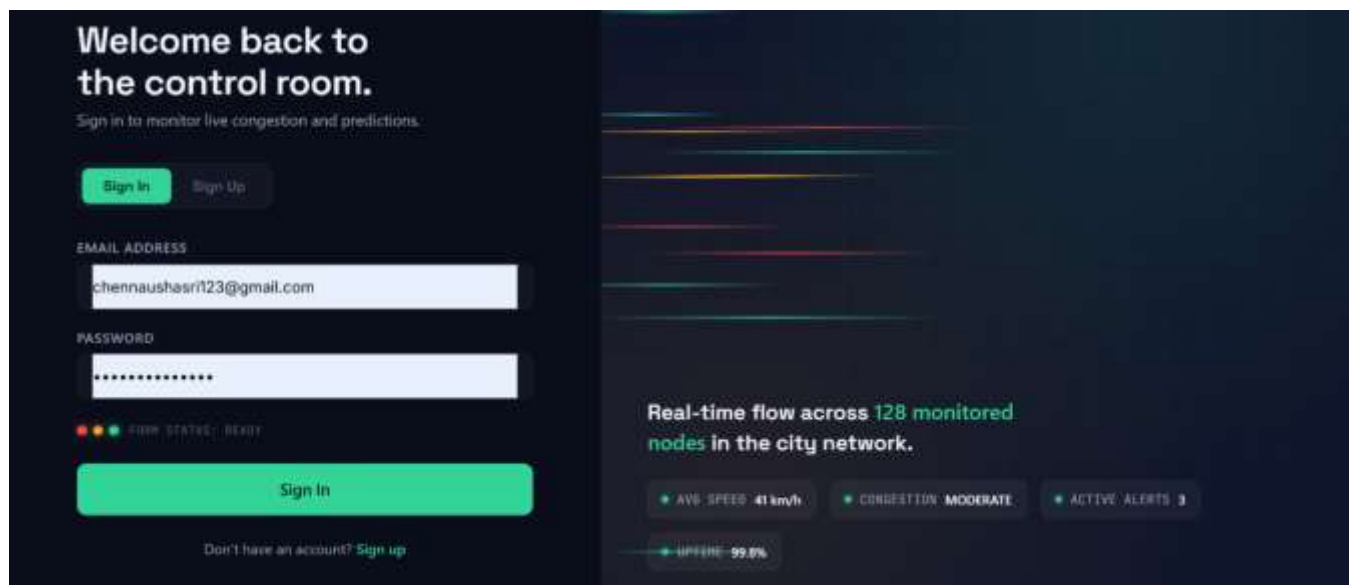
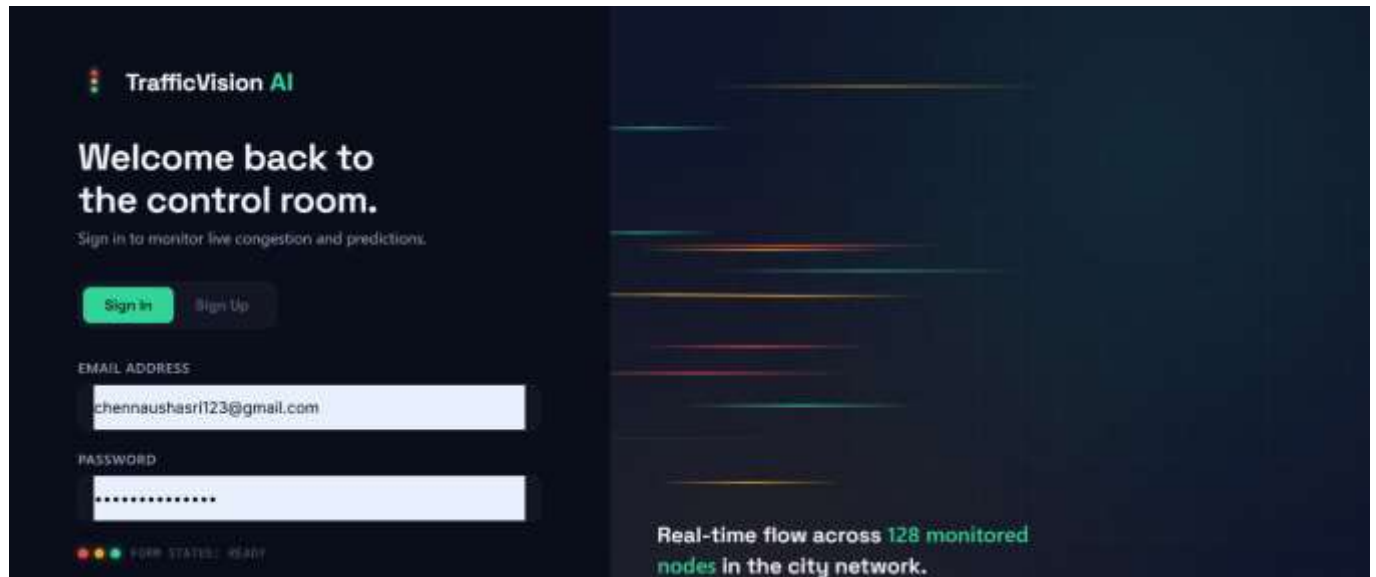




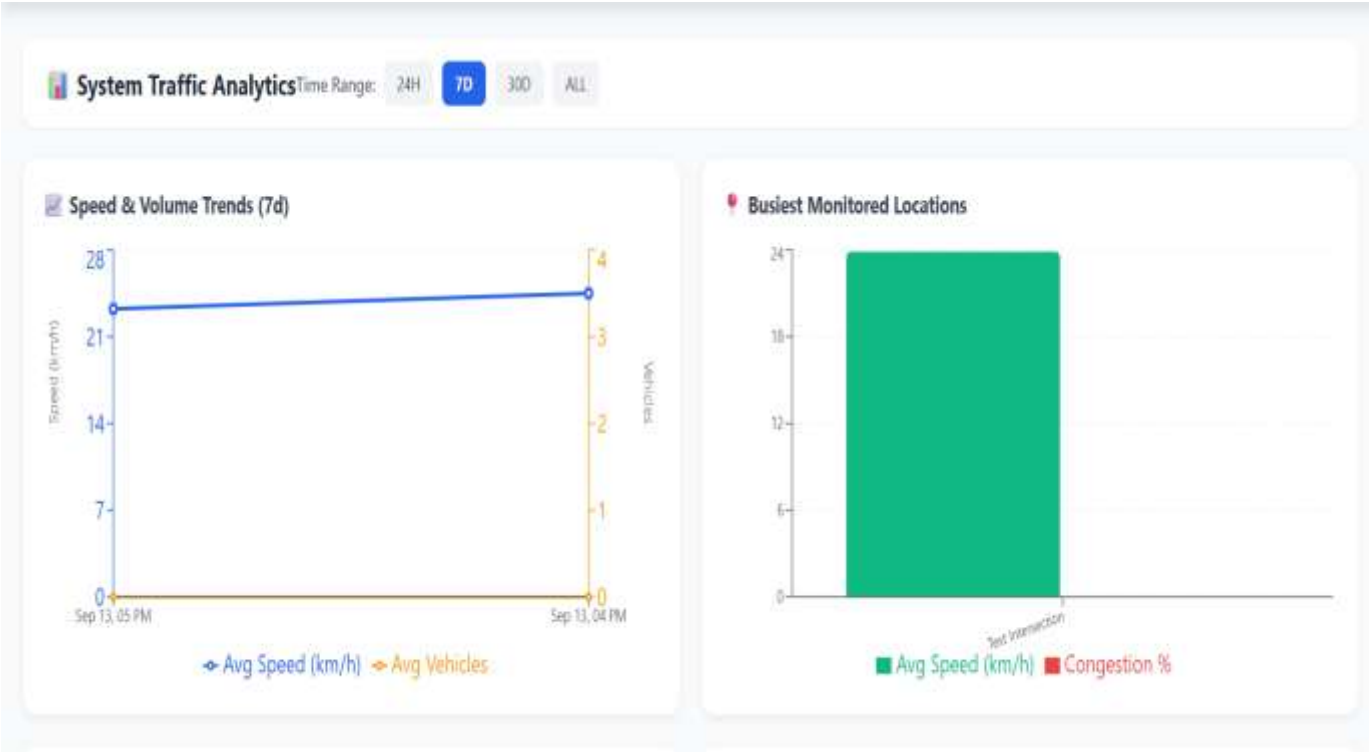
8.5 Workflow Implementation

The TrafficVision AI workflow begins by collecting historical and live traffic data. The collected data is then cleaned and preprocessed, followed by feature selection and loading of the prediction model. The system predicts the current congestion level and checks whether the congestion is high. If high congestion is detected, an alert is generated for monitoring; otherwise, the system continues normal processing. The dashboard is then updated with the latest traffic and prediction information, and the prediction result is stored in the database. This workflow provides a continuous process for monitoring traffic conditions, detecting congestion, generating alerts, and maintaining prediction record.

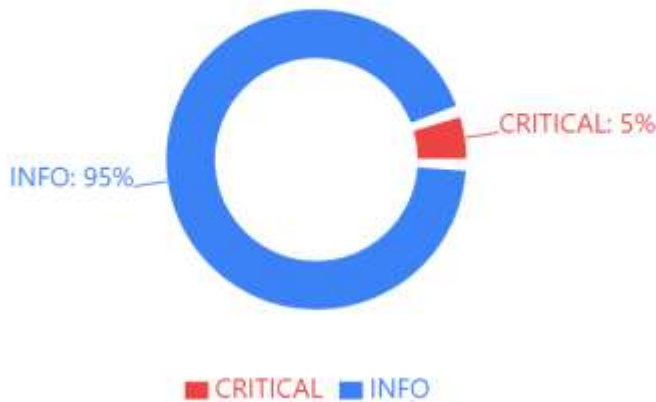
9 User Interface



Dashboards:

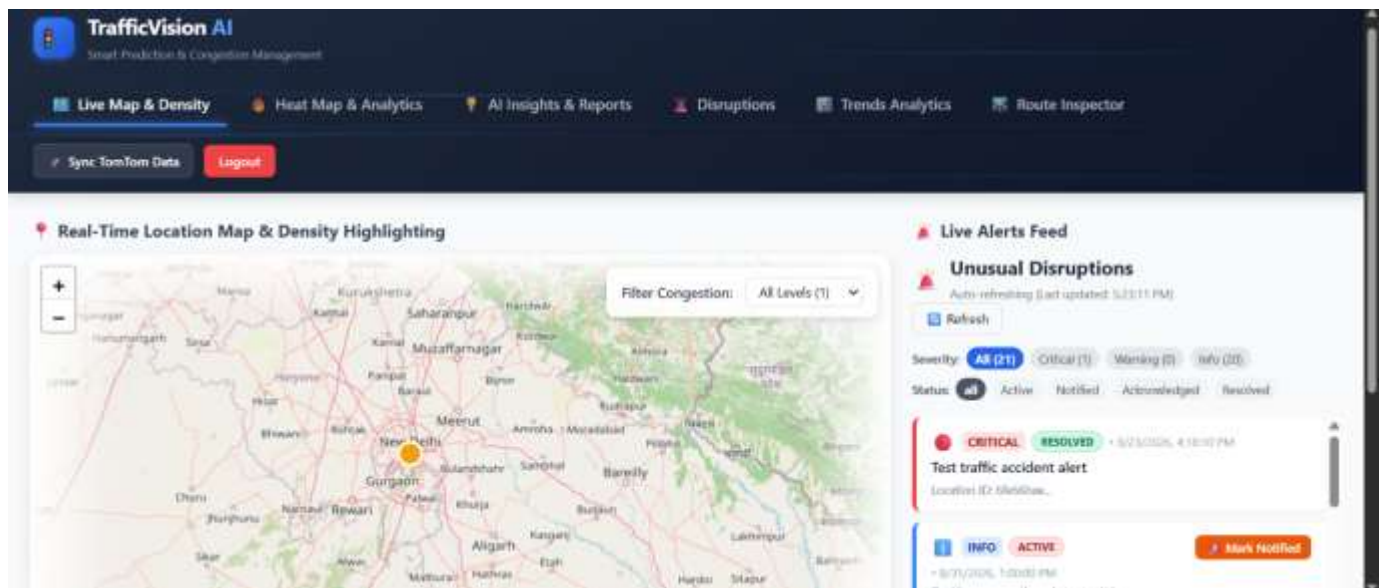


Disruption Alerts Breakdown



Dashboard: The dashboard provides a centralized view of the TrafficVision AI system. It presents important traffic information and system indicators in an organized interface, allowing users to quickly understand current traffic conditions and access different system modules.

Live Traffic Map:



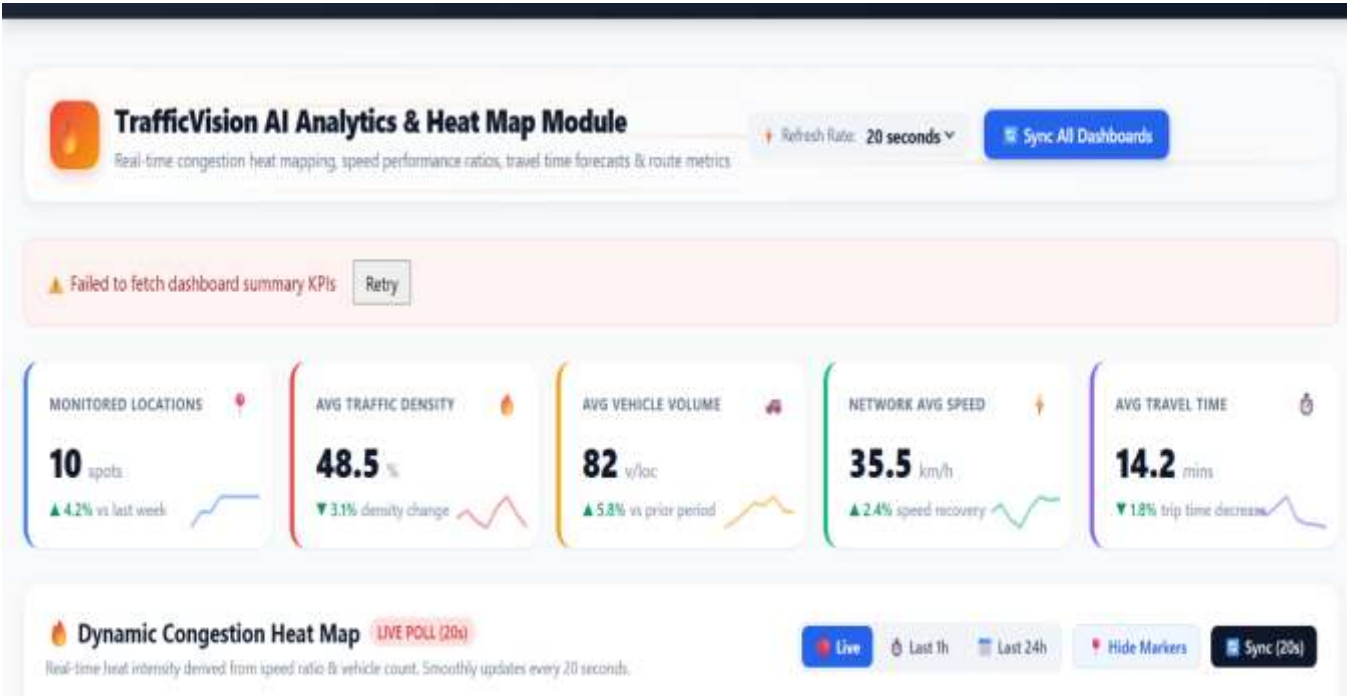
Live Traffic Map: The live map provides a visual representation of current traffic conditions across monitored locations. It helps users identify traffic congestion and affected areas geographically and supports quick understanding of traffic conditions at different road locations.

Alerts



Alerts Screen: The alerts interface displays traffic-related alerts generated by the system, including congestion and incident notifications, allowing users to monitor active, acknowledged, notified, and resolved events.

Analytics &Heat Map Screen



Analytics &Heat Map Module:The analytics interface presents traffic information through charts, graphs, and statistical indicators. It helps users analyze traffic patterns, compare traffic conditions, and understand historical and current traffic trends for better decision-making.



10 GitHub & Version Control

10.1 Repository

https://github.com/springboardmentor01092s-tech/Team_1_Smart_Traffic_AI.git

10.2 Repository Structure

Team_1_Smart_Traffic_AI/

```
|
|— config/
|   └─ db.js
|
|— controllers/
|   └─ trafficController.js
|   └─ routeController.js
|
|— database/
|   └─ trafficvision_schema.sql
|
|— routes/
|   └─ auth.js
|   └─ alertRoutes.js
|   └─ predictionRoutes.js
|   └─ routeRoutes.js
|
|— utils/
|   └─ utility files
|
|— src/
|   └─ frontend source files
|
|— public/
|   └─ frontend assets
|
|— AI/ML Processing/
|   └─ preprocess_traffic_data.py
|   └─ train_test_split_pipeline.py
|   └─ train_baseline_model.py
|   └─ evaluate_overfitting.py
|   └─ generate_traffic_report.py
|
|— server.js
|— package.json
|— package-lock.json
└─ .gitignore
```

The repository is organized into configuration, backend controllers, database, API routes, utility functions, frontend source files, and AI/ML processing scripts. This modular structure separates the major components of the TrafficVision AI system and supports easier development, testing, maintenance, and future enhancement.

10.3 Version Control

Git and GitHub were used for version control throughout the project. Development work was maintained using separate branches for feature development, while the main branch was used for the main project code. Changes were committed and pushed to GitHub to maintain project history and support team collaboration.

The project repository contains the frontend, backend, database schema, and AI/ML components required for the TrafficVision AI system.

11 Error Handling & Security

11.1 Handling

- Input Validation: Invalid API inputs are validated before processing.
- API Errors: Failed requests return appropriate HTTP status codes and error messages.
- Database Errors: Database exceptions are handled using backend error handling, with transactions rolled back when required.
- AI/ML Errors: Errors during traffic/congestion processing are handled without terminating the application.
- Authentication Errors: Invalid credentials or authentication tokens are handled appropriately.
- Congestion Handling: Traffic conditions are classified into LOW, MEDIUM, and HIGH levels, with appropriate actions such as normal monitoring, warnings, or alerts/alternate-route actions.

11.2 Security

11.2.1 API Gateway handles request routing, rate limiting, and security validation.

11.2.2 JWT Authentication: Used to authenticate users and protect authorized functionality.

11.2.3 Password Security: User passwords are hashed using bcrypt before being stored.

11.2.4 Environment Variables: Database credentials and authentication secrets are stored using environment variables.

11.2.5 Input Validation: API requests are validated before processing.

11.2.6 Data Protection: Sensitive credentials and tokens are not exposed in application responses or logs.

12 Testing

The TrafficVision AI system was tested at both the API and application levels to verify the functionality of the implemented features. API endpoints were tested using Postman and Newman, covering authentication, traffic-data retrieval, predictions, route analysis, travel-time estimation, alerts, and analytics. The API test collection achieved 20/20 successful assertions against the live backend.

Frontend functionality was tested using Vitest, covering the map, alerts, and analytics components, with 8/8 tests passing. The complete alert workflow was also tested through Active → Acknowledged → Resolved, and the corresponding status changes were verified in the PostgreSQL database. These tests helped confirm the reliability and correct integration of the major system workflows.

- **API Testing:** Postman/Newman collection tested the major backend endpoints.
- **API Test Result: 20/20 assertions passed** against the live backend.
- **Frontend Unit Testing:** Vitest was used to test the map, alerts, and analytics components.
- **Frontend Test Result:** 8/8 tests passed.
- **Workflow Testing:** Traffic-data retrieval, predictions, alerts, route analysis, travel-time, and analytics workflows were tested.
- **Alert Lifecycle Testing:** Verified Active → Acknowledged → Resolved status updates and database persistence.
- **AI/ML Validation:** The Random Forest model achieved 89.3% test accuracy and 0.79 macro F1-score.

13 RESULT

13.1 AI/ML Results

- **Model:** Random Forest Classifier
- **Dataset:** Real Bangalore traffic data
- **Accuracy:** 89.3%
- **Macro F1-Score:** 0.79
- **Overfitting:** Reduced using regularization techniques.
- **Congestion Classes:** Low, Moderate, High, and Severe
- **Overall Result:** The model provides effective congestion classification and supports the system's traffic prediction workflow

13.2 System Result

The TrafficVision AI system was evaluated based on API functionality, database integration, alert management, congestion monitoring, analytics, and frontend dashboard functionality.

API Testing: 20/20 Newman assertions passed.

Frontend Testing: 8/8 Vitest tests passed.

Alert Workflow: Active → Acknowledged → Resolved successfully verified.

Database Integration: Traffic and alert data successfully stored and retrieved through PostgreSQL.

Application Functionality: Dashboard, live traffic monitoring, analytics, alerts, predictions, and route-analysis features were tested successfully.

Response/Inference Time: No formal numerical measurements were recorded.

14 Deployment

The TrafficVision AI platform was developed and tested as a web-based application using a React/Vite frontend, Node.js/Express backend, and PostgreSQL database. During development, the frontend and backend were run locally and connected through REST APIs for communication and data exchange.

The backend manages API requests, authentication, traffic data, predictions, routes, and alerts, while PostgreSQL stores the application's data. Environment variables were used for database and application configuration to keep sensitive settings separate from the source code.

The complete system was tested in the local development environment with the frontend, backend, and database working together. The application workflows were verified through API testing and frontend testing to ensure that the major implemented features functioned correctly

Frontend: React.js with Vite

Backend: Node.js with Express.js

Database: PostgreSQL

API Communication: REST APIs

Configuration: Environment variables for sensitive settings

Testing Environment: Local integrated environment with frontend, backend, and database connected.

15 Challenges & Limitations

- Frontend–backend integration required configuration adjustments during development.
- PostgreSQL integration required repeated testing of database operations and API workflows.
- Implementing the complete alert lifecycle required coordination between the backend, database, and dashboard.
- The ML model required regularization to reduce overfitting.
- Route recommendation testing was limited by the availability of route/corridor data.
- Completing development, testing, and documentation within the project timeline was challenging.
- API Integration: Connecting multiple backend endpoints and ensuring consistent communication between frontend and backend required additional testing.
- Dependency Setup: Setting up the required Node.js packages and development environment was necessary before the application could run correctly.
- Testing and Debugging: Identifying and resolving integration issues during API and workflow testing required repeated debugging.
- Data Availability: Some features depended on the availability of suitable traffic and route/corridor data for testing.

16 Future Scope

- Integration with live traffic signal control systems for adaptive signal timing.
- Real-time vehicle counting using CCTV and computer vision.
- Mobile application for personalized route and congestion alerts.
- Expansion to multiple cities with city-specific model tuning.
- Further improvement of AI-based traffic prediction and route recommendations.
- Integration **with real-time traffic APIs and IoT sensors** for continuous traffic data.
- Improved **travel-time prediction** using larger and more diverse traffic datasets.
- Advanced **AI models** for **better congestion** and traffic-flow forecasting.
- Integration of **weather and road-condition data** for more accurate predictions.
- Real-time notifications for commuters and **traffic authorities** during major disruptions.
- Cloud-based deployment for **scalable multi-city traffic monitoring**.
- Integration with **IoT-based traffic sensors** for continuous traffic monitoring.
- Use of **weather and road-condition data** to improve congestion prediction accuracy.
- Development of **real-time travel-time and delay prediction** for monitored routes.
- Improved AI-based alternate route recommendations using live traffic conditions.
- Real-time mobile/web notifications for commuters during major traffic disruptions.
- Integration with emergency services to provide faster incident response and traffic clearance.
- Support for **historical traffic analysis** to identify recurring congestion patterns and peak-hour trends.
- Scalable deployment for **larger traffic networks** and **multiple cities**.

17 Conclusion

TrafficVision AI successfully combines real-time traffic monitoring, congestion prediction, route analysis, analytics, and alert management into a centralized web application. The system provides an integrated platform for monitoring traffic conditions and supporting traffic-management decisions to address challenges in traffic monitoring, congestion detection, prediction, route analysis, analytics, and alert management.

- Key achievements:
- Implemented traffic monitoring and congestion management workflows.
- Developed a Random Forest congestion prediction model.
- Achieved 89.3% test accuracy and 0.79 macro F1-score.
- Implemented the Active → Acknowledged → Resolved alert lifecycle.
- Successfully tested backend APIs with 20/20 Newman assertions.
- Successfully tested frontend components with 8/8 Vitest tests.
- Integrated traffic data, predictions, alerts, analytics, and route-related functionality.
- Overall, the project successfully achieved its core objectives and demonstrates the practical use of AI and web technologies for smart traffic management. The platform can be further enhanced through multi-city deployment, improved route recommendations, real-time vehicle detection, and integration with additional traffic and environmental data sources

18 References

- Traffic datasets: data.gov.in, OpenCity, Kaggle, Zenodo.
- Frameworks & libraries: FastAPI, Flask, React.js, Next.js, Scikit-learn, TensorFlow, Pandas, NumPy
- APIs: Google Maps API, OpenStreetMap API, Traffic APIs.